



Universitatea Tehnică de Construcții București

**Facultatea
de Hidrotehnică**

Proiect Inteligență Artificială

Predicție Prețuri Imobiliare din București

Student: Proaspătu Nicolae-Bogdan

Specializare: Automatică și Informatică Aplicată

An: IV

Cuprins

1. Introducere.....	3
1.1. Descrierea temei alese.....	3
1.2. Obiective propuse.....	3
1.3. Motivația alegerii temei.....	4
2. Tehnologii utilizate.....	5
2.1. Python.....	5
2.2. TensorFlow și Keras.....	5
2.3. Pandas și NumPy.....	5
2.4. Scikit-learn.....	6
2.5. Matplotlib și Seaborn.....	6
2.6. Tkinter.....	6
3. Implementarea Inteligenței Artificiale.....	7
3.1. Generarea datelor sintetice.....	7
3.2. Arhitectura rețelei neuronale.....	8
3.3. Procesul de antrenare.....	9
3.4. Metrice de evaluare.....	10
4. Realizarea proiectului.....	12
4.1. Interfața aplicației.....	12
4.2. Graficele de analiză.....	16
4.3. Mesajele din terminal.....	22
4.4. Fișierul CSV și structura datelor.....	23
5. Explicarea importurilor și bibliotecilor.....	24
6. Concluzie.....	26
7. Bibliografie.....	27

1. Introducere

1.1. Descrierea temei alese

Tema aleasă pentru acest proiect constă în realizarea unei aplicații de predicție a prețurilor imobiliare din București, utilizând tehnici de inteligență artificială, în special rețele neuronale artificiale. Aplicația, denumită UrbanPrice Bucharest, a fost dezvoltată folosind limbajul de programare Python împreună cu biblioteca TensorFlow/Keras pentru partea de machine learning și Tkinter pentru interfața grafică.

Aplicația oferă funcționalități esențiale pentru o platformă modernă de predicție imobiliară. Aceasta permite generarea de date sintetice realiste bazate pe piața din București, antrenarea unui model de rețea neuronală pentru predicție, vizualizarea graficelor de performanță și corelație, calculul prețului estimat pentru o proprietate pe baza caracteristicilor introduse de utilizator, precum și afișarea statisticilor detaliate despre piața imobiliară pe sectoare.

Tema este relevantă, actuală și se înscrie în categoria aplicațiilor de inteligență artificială cu aplicabilitate reală, în care utilizatorii pot interacționa cu un model de machine learning pentru a obține estimări de preț. Proiectul reflectă o abordare completă de tip end-to-end și demonstrează aplicarea bunelor practici în dezvoltarea de aplicații bazate pe inteligență artificială.

1.2. Obiective propuse

Proiectul are ca scop dezvoltarea unei aplicații moderne de predicție a prețurilor imobiliare din București, bazată pe inteligență artificială. Primul obiectiv major este generarea unui set de date sintetice realiste care să reflecte caracteristicile pieței imobiliare din București, luând în considerare factori precum sectorul, suprafața, anul construcției, distanța până la metrou, numărul de camere și băi.

Al doilea obiectiv constă în implementarea și antrenarea unei rețele neuronale pentru predicția prețurilor, utilizând arhitectura Sequential din Keras cu straturi Dense, BatchNormalization și Dropout. Următorul obiectiv vizează dezvoltarea unei interfețe grafice intuitive folosind Tkinter, care să permită utilizatorilor să introducă caracteristicile unei proprietăți și să obțină o estimare de preț.

De asemenea, proiectul își propune vizualizarea graficelor relevante, incluzând evoluția antrenării, distribuția prețurilor pe sectoare, matricea de corelație, relația preț-suprafață și acuratețea predicțiilor. Nu în ultimul rând, un obiectiv important este evaluarea performanței modelului folosind metrici standard precum R^2 Score, MAE și MSE.

1.3. Motivația alegerii temei

Am ales să dezvoltăm o aplicație de predicție a prețurilor imobiliare deoarece domeniul imobiliar este unul dinamic, de actualitate și cu o mare aplicabilitate practică. Prețurile proprietăților din București variază semnificativ în funcție de mulți factori precum sectorul, suprafața, vechimea clădirii și proximitatea față de stațiile de metrou, ceea ce face acest domeniu ideal pentru aplicarea tehnicilor de machine learning.

În plus, un astfel de proiect permite implementarea unei game variate de tehnici de inteligență artificială relevante, precum preprocesarea datelor, normalizarea cu StandardScaler, arhitectura rețelelor neuronale cu regularizare L2 și dropout, optimizarea cu Adam și learning rate scheduling, early stopping pentru prevenirea overfitting-ului și evaluarea cu metrici multiple.

Totodată, proiectul oferă o bună oportunitate de consolidare a cunoștințelor de Python, TensorFlow, Keras, Pandas, NumPy și Matplotlib în contextul unei aplicații reale și utile. Alegerea acestei teme reflectă dorința mea de a crea o aplicație funcțională, modernă și bine structurată, care să demonstreze capacitatea de a rezolva probleme concrete din lumea reală folosind inteligența artificială.

2. Tehnologii utilizate

Pentru dezvoltarea aplicației de predicție a prețurilor imobiliare, am folosit un set complet de tehnologii și biblioteci Python esențiale pentru machine learning și data science. Acestea acoperă toate componentele necesare pentru o aplicație modernă, de la procesarea datelor până la construirea și antrenarea modelului de rețea neuronală, vizualizarea rezultatelor și interfața grafică.

2.1. Python

Python este un limbaj de programare interpretat, de nivel înalt, cu tipizare dinamică. Este cel mai popular limbaj pentru aplicațiile de machine learning și data science datorită sintaxei sale clare și a ecosistemului vast de biblioteci specializate. Python oferă o curbă de învățare accesibilă și permite dezvoltarea rapidă a prototipurilor.

În cadrul acestui proiect, Python servește drept fundament pentru întreaga aplicație, facilitând integrarea multiplelor biblioteci. TensorFlow este folosit pentru rețele neuronale, Pandas pentru manipularea datelor, Matplotlib pentru vizualizări și Tkinter pentru interfața grafică. Versiunea utilizată este Python 3.11, care oferă performanțe îmbunătățite și suport complet pentru toate bibliotecile utilizate.

2.2. TensorFlow și Keras

TensorFlow este o platformă open-source pentru machine learning dezvoltată de Google. Este una dintre cele mai utilizate biblioteci pentru construirea și antrenarea rețelelor neuronale, oferind suport pentru calcul pe GPU, deployment în producție și o gamă largă de instrumente pentru cercetare și dezvoltare.

Keras este o interfață de programare de nivel înalt pentru rețele neuronale, integrată în TensorFlow începând cu versiunea 2.0. Keras simplifică procesul de construire a modelelor prin API-ul său intuitiv, permițând definirea arhitecturii rețelei folosind straturi predefinite precum Dense, BatchNormalization și Dropout. În acest proiect, am utilizat modelul Sequential din Keras pentru a construi o rețea neuronală cu arhitectură feedforward, incluzând straturi Dense cu activare ReLU, regularizare L2, BatchNormalization și Dropout, toate optimizate cu algoritmul Adam.

2.3. Pandas și NumPy

Pandas este o bibliotecă Python pentru manipularea și analiza datelor. Oferă structuri de date puternice precum DataFrame și Series, care permit lucrul eficient cu date tabelare. Pandas facilitează operații precum filtrarea, gruparea, agregarea și transformarea datelor. În acest proiect, Pandas este utilizat pentru crearea și manipularea DataFrame-ului cu datele sintetice, calcularea statisticilor pe sectoare și exportul datelor în format CSV.

NumPy, prescurtare de la Numerical Python, este biblioteca fundamentală pentru calcul numeric în Python. Oferă suport pentru array-uri multidimensionale și o colecție vastă de funcții matematice pentru operații pe aceste array-uri. NumPy este baza pe care sunt construite

majoritatea bibliotecilor de data science din Python. În acest proiect, NumPy este folosit pentru generarea distribuțiilor statistice în procesul de creare a datelor sintetice, precum și pentru operațiile matriciale necesare în procesarea datelor pentru rețeaua neuronală.

2.4. Scikit-learn

Scikit-learn este o bibliotecă Python pentru machine learning care oferă implementări eficiente ale algoritmilor de clasificare, regresie și clustering. Include, de asemenea, instrumente pentru preprocesarea datelor, selecția modelelor și evaluarea performanței.

În acest proiect, am utilizat din scikit-learn mai multe componente esențiale. StandardScaler este folosit pentru normalizarea features-urilor, transformând datele astfel încât să aibă media 0 și deviația standard 1. Funcția `train_test_split` împarte datele în seturi de antrenare, validare și test. De asemenea, funcțiile `r2_score`, `mean_absolute_error` și `mean_squared_error` sunt utilizate pentru evaluarea performanței modelului.

2.5. Matplotlib și Seaborn

Matplotlib este biblioteca standard pentru vizualizare în Python. Oferă o interfață flexibilă pentru crearea unei game largi de grafice, incluzând scatter plots, line plots, bar charts, histograme și heatmaps. Matplotlib permite personalizarea detaliată a fiecărui element vizual. Seaborn este o bibliotecă de vizualizare statistică construită pe Matplotlib, oferind teme estetice îmbunătățite și funcții specializate pentru vizualizarea relațiilor statistice.

În acest proiect, Matplotlib și Seaborn sunt folosite pentru generarea tuturor graficelor din aplicație, incluzând evoluția loss-ului în timpul antrenării, boxplot-uri pentru distribuția prețurilor pe sectoare, heatmap pentru matricea de corelație, scatter plot pentru relația preț-suprafață, bar chart pentru importanța features-urilor și scatter plot pentru predicții versus valori reale.

2.6. Tkinter

Tkinter este biblioteca standard pentru crearea interfețelor grafice în Python. Este inclusă în instalarea standard Python și oferă widget-uri pentru butoane, etichete, câmpuri de text, slider-uri, tab-uri și multe altele. Tkinter folosește toolkit-ul Tk pentru rendering-ul interfețelor.

În acest proiect, Tkinter este utilizat pentru construirea întregii interfețe grafice a aplicației. Interfața include un header cu titlul aplicației, un panou lateral cu tab-uri pentru predicție, statistici și istoric, slider-uri pentru introducerea caracteristicilor proprietății, butoane pentru calculul prețului, reantrenarea modelului și salvarea datelor, precum și un panou principal cu tab-uri pentru diferitele grafice de analiză.

3. Implementarea Inteligenței Artificiale

3.1. Generarea datelor sintetice

Un aspect fundamental al acestui proiect este generarea datelor sintetice pentru antrenarea modelului. Întrucât datele reale despre prețurile imobiliare din București nu sunt ușor accesibile în format structurat, am ales să generăm un set de date sintetice care să reflecte cât mai fidel caracteristicile reale ale pieței.

Funcția `generate_synthetic_data()` creează un DataFrame cu 2000 de înregistrări, fiecare reprezentând o proprietate imobiliară. Pentru fiecare proprietate sunt generate opt caracteristici principale. Sectorul este o valoare întreagă între 1 și 6, reprezentând cele șase sectoare ale Bucureștiului. Suprafața este generată folosind o distribuție normală cu media 70 și deviația standard 25, fiind limitată între 30 și 200 de metri pătrați. Numărul de camere variază între 1 și 5, iar etajul între 0 și 10.

Anul construcției este o valoare întreagă între 1970 și 2025, reflectând diversitatea fondului locativ din București. Distanța până la metrou folosește o distribuție exponențială cu parametrul 1.5, limitată între 0.1 și 5 kilometri, simulând faptul că majoritatea proprietăților sunt relativ aproape de stațiile de metrou, dar există și proprietăți în zone mai îndepărtate. Numărul de băi este fie 1, fie 2, cu probabilități de 65% respectiv 35%, iar balconul este o variabilă binară cu probabilitate de 70% să existe.

Calculul prețului sintetic

Prețul fiecărei proprietăți este calculat folosind o formulă complexă care ia în considerare mulți factori ai pieței imobiliare din București. Formula de bază înmulțește prețul de bază per metru pătrat cu suprafața, apoi aplică o serie de multiplicatori pentru vârstă, proximitatea față de metrou, etaj și număr de camere, adăugând la final diverse bonusuri.

Prețul de bază per metru pătrat variază în funcție de sector, reflectând ierarhia reală a prețurilor pe sectoare în București. Sectorul 1 are cel mai mare preț de bază, urmat de Sectorul 2, apoi Sectoarele 3, 4, 5 și 6 cu prețuri progresiv mai mici. Această stratificare corespunde realității pieței imobiliare bucureștene, unde zona centrală și de nord comandă prețuri premium.

Multiplicatorul pentru vârstă penalizează clădirile vechi și recompensează construcțiile noi. Clădirile cu vârsta sub 3 ani primesc un bonus semnificativ, cele între 3 și 8 ani un bonus moderat, iar cele foarte vechi, peste 35 de ani, suferă penalizări considerabile. Acest aspect reflectă preferința cumpărătorilor pentru construcții moderne cu finisaje și instalații la standarde actuale.

Multiplicatorul pentru distanța până la metrou este deosebit de important în București, unde transportul public joacă un rol crucial. Proprietățile situate la sub 300 de metri de o stație de metrou beneficiază de un bonus substanțial, în timp ce cele aflate la peste 3.5 kilometri suferă penalizări semnificative. Această variabilă captează valoarea adăugată a accesibilității la transportul în comun.

La final, pentru a simula variația naturală a prețurilor pe piață, se adaugă un zgomot gaussian cu media 1 și deviația standard 0.08. Acest zgomot reprezintă factorii aleatori care influențează prețurile în realitate, precum starea specifică a apartamentului, urgența vânzătorului sau condițiile de negociere.

3.2. Arhitectura rețelei neuronale

Modelul de rețea neuronală utilizat în acest proiect este o rețea feedforward de tip Sequential cu mai multe straturi Dense. Arhitectura a fost proiectată pentru a echilibra capacitatea de învățare cu prevenirea overfitting-ului, o problemă frecventă în machine learning când modelul învață prea bine datele de antrenare și nu generalizează bine pe date noi.

Rețeaua neuronală este construită ca o succesiune de straturi, fiecare cu un rol specific în procesarea informației. Primul strat Dense primește cele 8 features de intrare și le transformă într-o reprezentare cu 128 de dimensiuni, folosind funcția de activare ReLU și regularizare L2. Acest strat are rolul de a extrage primele features de nivel înalt din datele brute.

După fiecare strat Dense, urmează un strat de BatchNormalization care normalizează activările stratului anterior, apoi un strat de Dropout care dezactivează aleatoriu un procent din neuroni. Al doilea strat Dense reduce dimensionalitatea la 64 de neuroni, al treilea la 32, iar al patrulea la 16. Stratul final de output are un singur neuron fără funcție de activare, producând predicția de preț ca valoare continuă.

Regularizare L2

Regularizarea L2, cunoscută și sub numele de Ridge regularization, este o tehnică esențială pentru prevenirea overfitting-ului. Aceasta funcționează prin adăugarea la funcția de loss a unui termen proporțional cu suma pătratelor ponderilor rețelei. Practic, modelul este penalizat pentru ponderile mari, fiind încurajat să folosească ponderi mai mici și mai distribuite.

Intuiția din spatele acestei tehnici este simplă: ponderile foarte mari tind să facă modelul prea sensibil la variații mici în date, ducând la memorarea datelor de antrenare în loc de învățarea tiparelor generale. Prin limitarea magnitudinii ponderilor, regularizarea L2 forțează modelul să găsească soluții mai simple și mai robuste.

Batch Normalization

BatchNormalization este o tehnică care normalizează intrările fiecărui strat prin centrarea și scalarea lor. Această normalizare rezolvă problema internal covariate shift, adică modificarea distribuției intrărilor straturilor în timpul antrenării, pe măsură ce ponderile straturilor anterioare se schimbă.

Beneficiile BatchNormalization sunt multiple. În primul rând, permite o antrenare mai rapidă și mai stabilă. În al doilea rând, face posibilă utilizarea unor learning rate-uri mai mari fără riscul de divergență. În al treilea rând, are un efect moderat de regularizare, contribuind la prevenirea overfitting-ului.

Dropout

Dropout este o tehnică de regularizare care dezactivează aleatoriu un procent din neuroni în timpul fiecărui pas de antrenare. La fiecare iterație, un set diferit de neuroni este dezactivat, forțând rețeaua să dezvolte reprezentări mai robuste care nu depind de neuroni specifici.

Această tehnică poate fi înțeleasă ca o formă de ensemble learning implicit: în esență, antrenăm simultan multe subrețele diferite și le combinăm la inferență. Ratele de dropout folosite în proiect sunt descrescătoare pe măsură ce avansăm în rețea, permițând straturilor mai profunde să păstreze mai multe informații pentru predicția finală.

3.3. Procesul de antrenare

Procesul de antrenare a rețelei neuronale urmează o metodologie standard de machine learning, cu câteva optimizări specifice pentru îmbunătățirea performanței și prevenirea overfitting-ului.

Împărțirea datelor

Datele sunt împărțite în trei seturi distincte cu roluri diferite în procesul de machine learning. Setul de antrenare, reprezentând 70% din date, este folosit pentru ajustarea ponderilor rețelei. Setul de validare, reprezentând 15% din date, este folosit pentru monitorizarea performanței în timpul antrenării și pentru deciziile de early stopping. Setul de test, reprezentând tot 15% din date, este păstrat complet separat și folosit doar pentru evaluarea finală a modelului.

Această împărțire este esențială pentru a obține o estimare corectă a performanței modelului pe date nevăzute. Folosind `train_test_split` din `scikit-learn` cu un `random_state` fix, asigurăm reproducibilitatea experimentelor.

Normalizarea datelor

`StandardScaler` din `scikit-learn` este utilizat pentru normalizarea features-urilor. Scaler-ul este antrenat doar pe datele de antrenare, calculând media și deviația standard pentru fiecare feature, și apoi aplicat pe toate seturile de date. Această abordare previne data leakage, adică scurgerea de informații din seturile de validare și test în procesul de antrenare.

Normalizarea transformă fiecare feature astfel încât să aibă media 0 și deviația standard 1. Acest lucru este important deoarece features-urile au scale foarte diferite în datele noastre: suprafața variază de la 30 la 200, în timp ce balconul este doar 0 sau 1. Fără normalizare, features-urile cu valori mari ar domina procesul de învățare.

Optimizator și Learning Rate

Optimizatorul folosit este Adam, prescurtare de la Adaptive Moment Estimation. Adam combină beneficiile a două tehnici anterioare: AdaGrad, care adaptează learning rate-ul pentru fiecare parametru în funcție de frecvența actualizărilor, și RMSprop, care normalizează gradientii folosind o medie mobilă exponențială. Rezultatul este un optimizator robust care funcționează bine pentru majoritatea problemelor.

Learning rate-ul utilizează un schedule exponențial descrescător. Această abordare permite o convergență rapidă la început, când modelul este departe de optim, și o ajustare fină către final, când se apropie de soluția optimă. Practic, modelul face pași mari la început pentru a explora spațiul soluțiilor, apoi pași din ce în ce mai mici pentru a se stabili într-un minim.

Callbacks

EarlyStopping este un callback care monitorizează validation loss și oprește antrenarea dacă nu există îmbunătățire timp de un număr specificat de epoci consecutive. La oprire, se restaurează ponderile de la cea mai bună epocă. Acest callback previne overfitting-ul și economisește timp de calcul, oprind antrenarea când modelul nu mai învață nimic util.

ReduceLRonPlateau este un callback care reduce learning rate-ul când validation loss stagnează. Când modelul ajunge pe un platou și nu mai progresează, reducerea learning rate-ului îi permite să facă ajustări mai fine și să scape din minime locale. Combinația acestor două callbacks asigură o antrenare eficientă și robustă.

3.4. Metrici de evaluare

Pentru evaluarea performanței modelului de regresie, am utilizat trei metrice standard care oferă perspective complementare asupra calității predicțiilor. Aceste metrice sunt calculate după fiecare antrenare a modelului și afișate atât în terminal, cât și pe graficul de predicții versus realitate.

R² Score (Coeficientul de determinare)

R² Score măsoară proporția varianței din variabila dependentă, în cazul nostru prețul, care este explicată de variabilele independente, adică features-urile proprietății. Valoarea R² variază teoretic de la minus infinit la 1, unde 1 înseamnă predicție perfectă, 0 înseamnă că modelul prezice doar media, iar valori negative indică un model mai rău decât prezicerea constantă a mediei.

În practică, pentru probleme de regresie pe date reale, un R² peste 0.9 este considerat foarte bun, indicând că modelul explică peste 90% din variația prețurilor. Valoarea exactă obținută variază de la o antrenare la alta, în funcție de inițializarea aleatoare a ponderilor și de split-ul datelor, dar în general modelul nostru atinge valori ridicate ale acestui indicator.

MAE (Mean Absolute Error)

MAE calculează media valorilor absolute ale diferențelor dintre predicții și valorile reale. Această metrică este ușor de interpretat deoarece este exprimată în aceeași unități ca variabila țintă, în cazul nostru Euro. MAE ne spune, în medie, cu cât diferă predicțiile modelului de prețurile reale.

Avantajul MAE față de alte metrice este robustețea sa la outlieri. Deoarece nu ridică diferențele la pătrat, erorile mari nu domină calculul. În contextul prețurilor imobiliare, care pot varia de la zeci de mii la sute de mii de euro, o eroare medie de câteva mii sau zeci de mii de euro poate fi considerată acceptabilă, în funcție de intervalul de prețuri din date.

MSE (Mean Squared Error)

MSE calculează media pătratelor diferențelor dintre predicții și valorile reale. Această metrică penalizează mai puternic erorile mari datorită ridicării la pătrat, făcând-o sensibilă la outlieri.

MSE este utilizat ca funcție de loss în timpul antrenării deoarece este derivabilă și convexă, facilitând optimizarea prin gradient descent. Rădăcina pătrată a MSE, numită RMSE, este adesea raportată pentru interpretabilitate, fiind în aceleași unități ca variabila țintă. Valorile concrete ale acestor metrici depind de antrenarea specifică și de caracteristicile datelor generate.

4. Realizarea proiectului

4.1. Interfața aplicației

Interfața aplicației UrbanPrice Bucharest a fost dezvoltată folosind biblioteca Tkinter și oferă o experiență utilizator modernă și intuitivă. Designul folosește o schemă de culori profesională cu alb dominant și accente de roșu, creând o identitate vizuală distinctivă.

La deschiderea aplicației, utilizatorul este întâmpinat de o interfață împărțită în două panouri principale. Panoul din stânga este destinat introducerii datelor și predicției, în timp ce panoul din dreapta afișează graficele și statisticile.

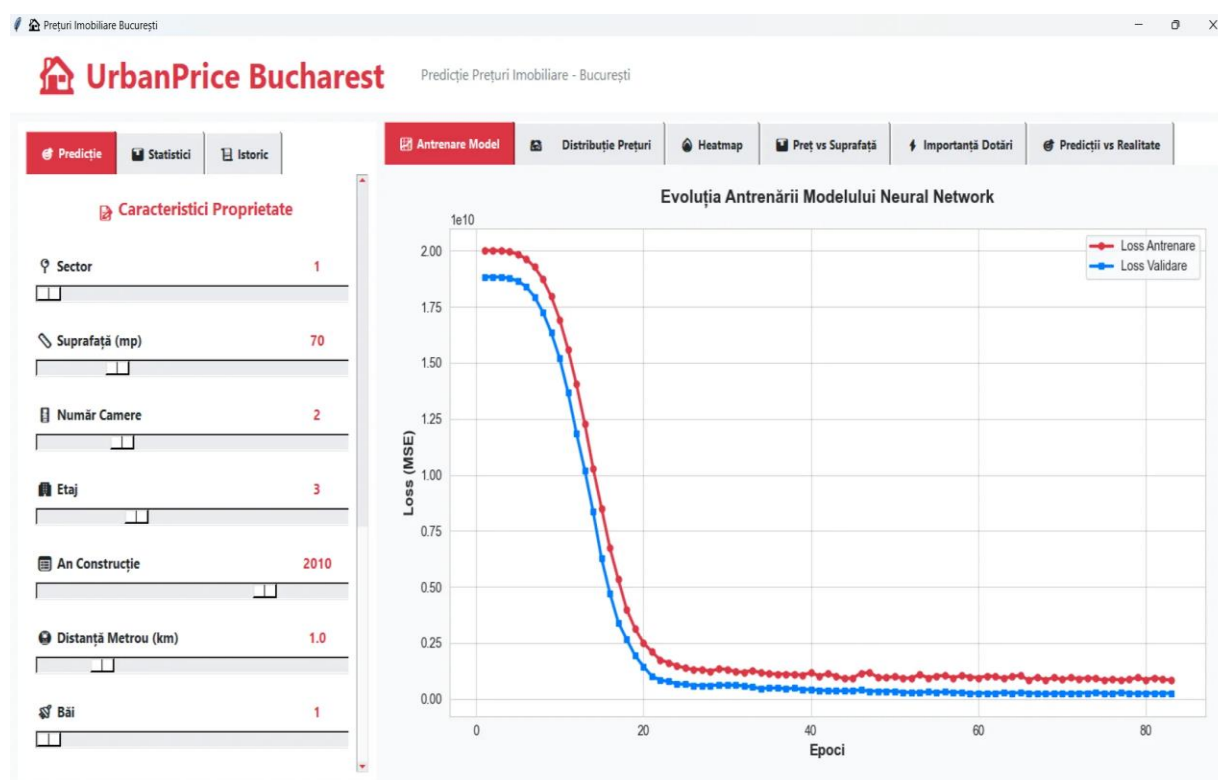


Fig. 1 Interfața principală a aplicației UrbanPrice Bucharest

Panoul din stânga conține trei tab-uri principale. Tab-ul Predicție permite introducerea caracteristicilor proprietății și calculul prețului estimat. Tab-ul Statistici afișează statisticile pieței imobiliare pe sectoare. Tab-ul Istoric păstrează un jurnal al predicțiilor efectuate în sesiunea curentă.

În tab-ul de predicție, utilizatorul poate ajusta caracteristicile proprietății folosind slider-uri intuitive. Sunt disponibile opt parametri: Sector, Suprafața în metri pătrați, Numărul de Camere, Etajul, Anul Construcției, Distanța până la Metrou în kilometri, Numărul de Băi și prezența Balconului. Fiecare slider afișează valoarea curentă în timp real, permițând ajustări fine.

Rezultatul predicției

După apăsarea butonului CALCULEAZĂ PREȚ, aplicația afișează rezultatul predicției într-un format clar și informativ. Rezultatul include prețul estimat atât în Euro cât și în RON, prețul pe metru pătrat, comparația cu media sectorului respectiv și comparația cu media generală a pieței.

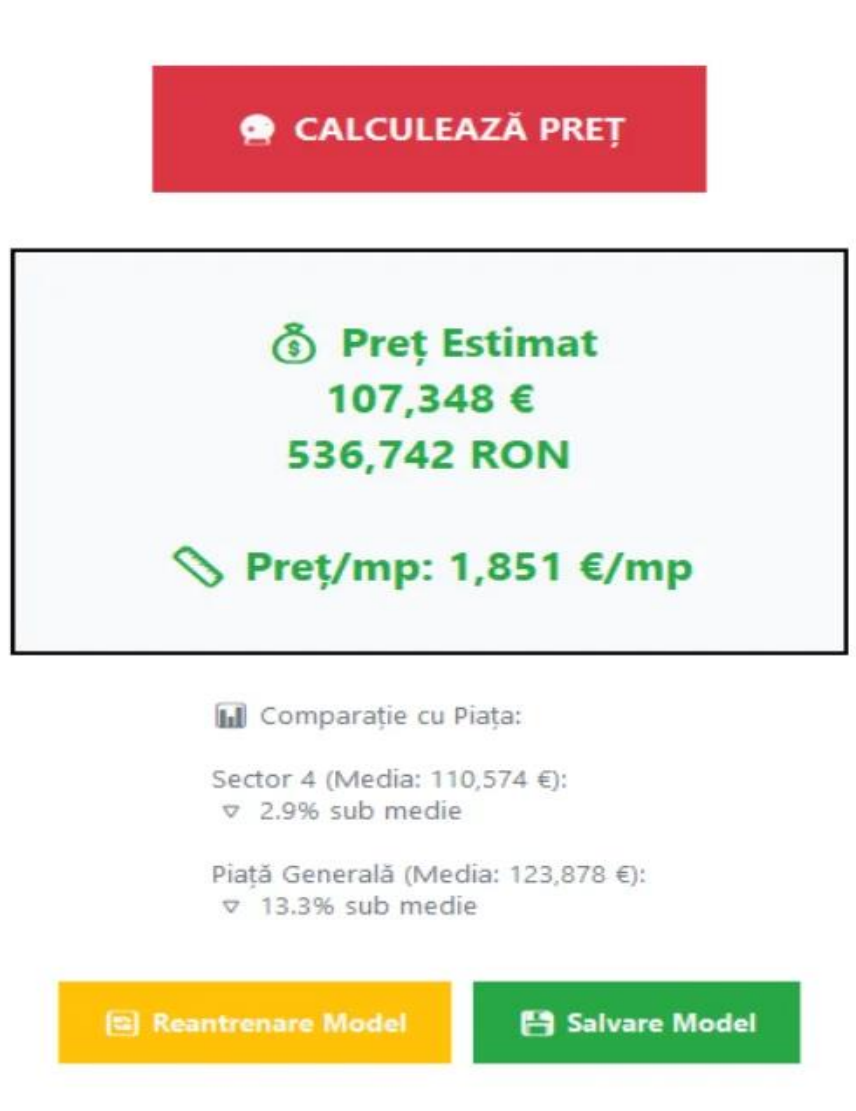


Fig. 2 Rezultatul predicției cu comparație de piață

Această prezentare contextuală a rezultatului ajută utilizatorul să înțeleagă nu doar valoarea estimată a proprietății, ci și poziționarea acesteia în raport cu piața. Dacă prețul estimat este sub media sectorului, utilizatorul știe că poate fi o oportunitate bună de achiziție, în timp ce un preț peste medie poate indica o proprietate premium sau potențial supraevaluată.

Tab-ul Statistici

Tab-ul Statistici oferă o vedere detaliată asupra datelor pieței imobiliare analizate. Sunt afișate statistici generale precum numărul total de proprietăți din setul de date, prețul mediu pe piață, prețul mediu pe metru pătrat și intervalul de prețuri.

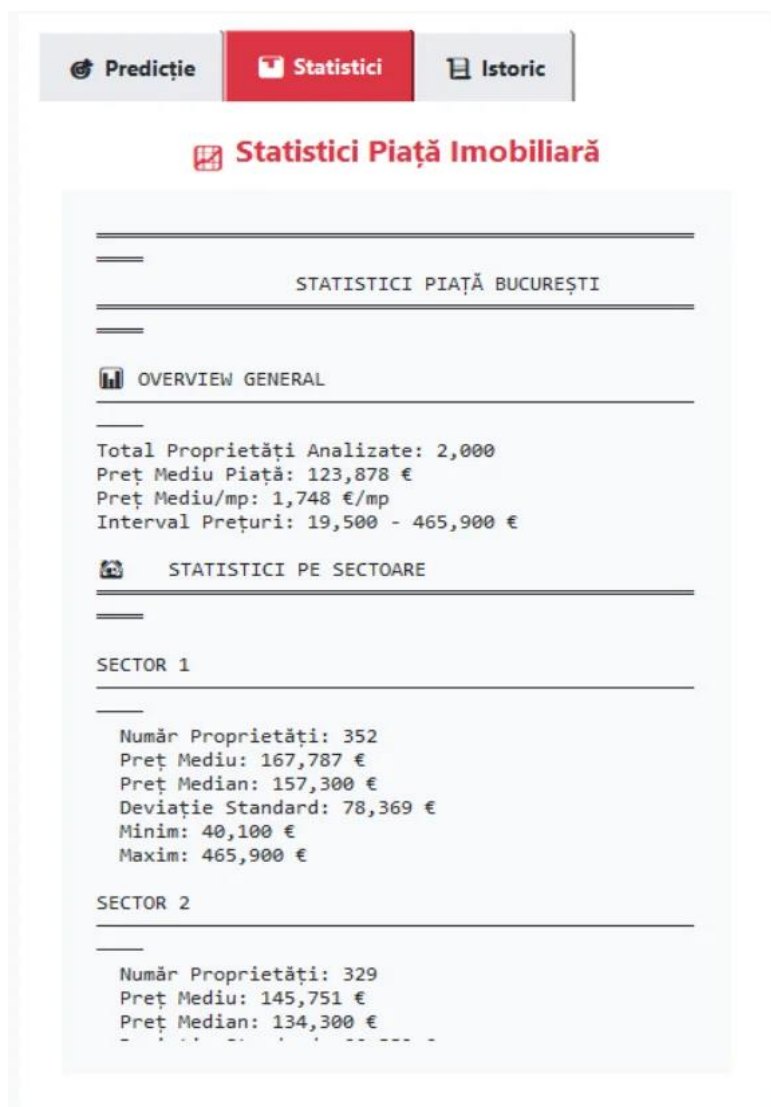


Fig. 3 Tab-ul Statistici cu date despre piața imobiliară

Pentru fiecare sector sunt afișate statistici detaliate incluzând numărul de proprietăți, prețul mediu, prețul median, deviația standard, prețul minim și prețul maxim. Aceste informații ajută utilizatorul să înțeleagă contextul pieței pentru fiecare zonă și să compare diferitele sectoare ale Bucureștiului.

Tab-ul Istoric

Tab-ul Istoric păstrează un jurnal al tuturor predicțiilor efectuate în sesiunea curentă. Fiecare intrare include timestamp-ul predicției, prețul estimat în Euro și RON, sectorul, suprafața, prețul pe metru pătrat, numărul de camere, etajul și anul construcției.

The screenshot displays the 'Istoric' tab, which contains a list of prediction records. Each record is separated by a dashed line and includes the following information:

- Timestamp: [2026-01-12 21:35:07]
- Price: Preț: 100,819 € (504,093 RON)
- Sector: Sector: 5 | Suprafață: 69mp | Preț/mp: 1,461 €
- Rooms: Camere: 4 | Etaj: 1 | An: 2000

Below the first record, there are four more records with similar details, including prices in both EUR and RON, and various property attributes like sector, area, price per square meter, number of rooms, floor, and year.

At the bottom of the interface, there are two buttons: a red button labeled 'Șterge Istoric' (Delete History) and a blue button labeled 'Export CSV'.

Fig. 4 Tab-ul Istoric cu predicțiile anterioare

Utilizatorul poate șterge istoricul sau îl poate exporta în format CSV pentru analiză ulterioară. Funcția de export creează un fișier CSV cu toate detaliile predicțiilor, util pentru compararea mai multor proprietăți sau pentru documentarea procesului de căutare.

4.2. Graficele de analiză

Panoul din dreapta al aplicației conține șase tab-uri cu grafice care oferă perspective diferite asupra datelor și performanței modelului. Aceste vizualizări sunt esențiale pentru înțelegerea comportamentului modelului și a caracteristicilor pieței imobiliare.

Graficul Antrenare Model

Primul grafic prezintă evoluția loss-ului, adică a erorii, în timpul antrenării rețelei neuronale. Sunt afișate două curbe: Loss Antrenare și Loss Validare. Graficul permite vizualizarea convergenței modelului și identificarea eventualelor probleme de overfitting.

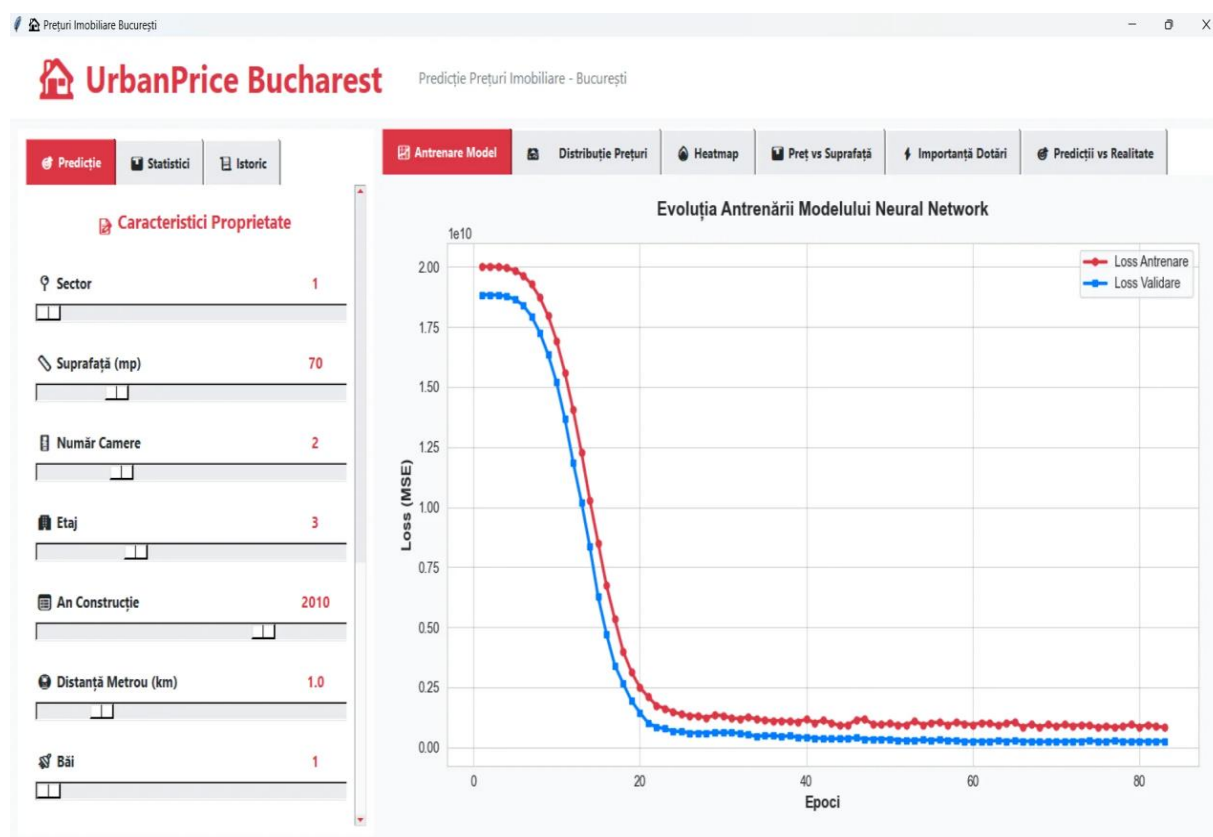


Fig. 5 Evoluția antrenării modelului Neural Network

Într-o antrenare sănătoasă, ambele curbe scad rapid în primele epoci, apoi converg către o valoare stabilă. Faptul că cele două curbe rămân apropiate indică absența overfitting-ului. Dacă curba de validare ar începe să crească în timp ce cea de antrenare continuă să scadă, ar fi un semn clar de overfitting, moment în care early stopping oprește antrenarea.

Graficul Distribuție Prețuri

Al doilea grafic este un boxplot care prezintă distribuția prețurilor pe fiecare sector al Bucureștiului. Boxplot-urile arată mediana reprezentată prin linia continuă din interiorul cutiei, media prin linia întreruptă, quartilele prin marginile cutiei și valorile aberante prin puncte individuale.

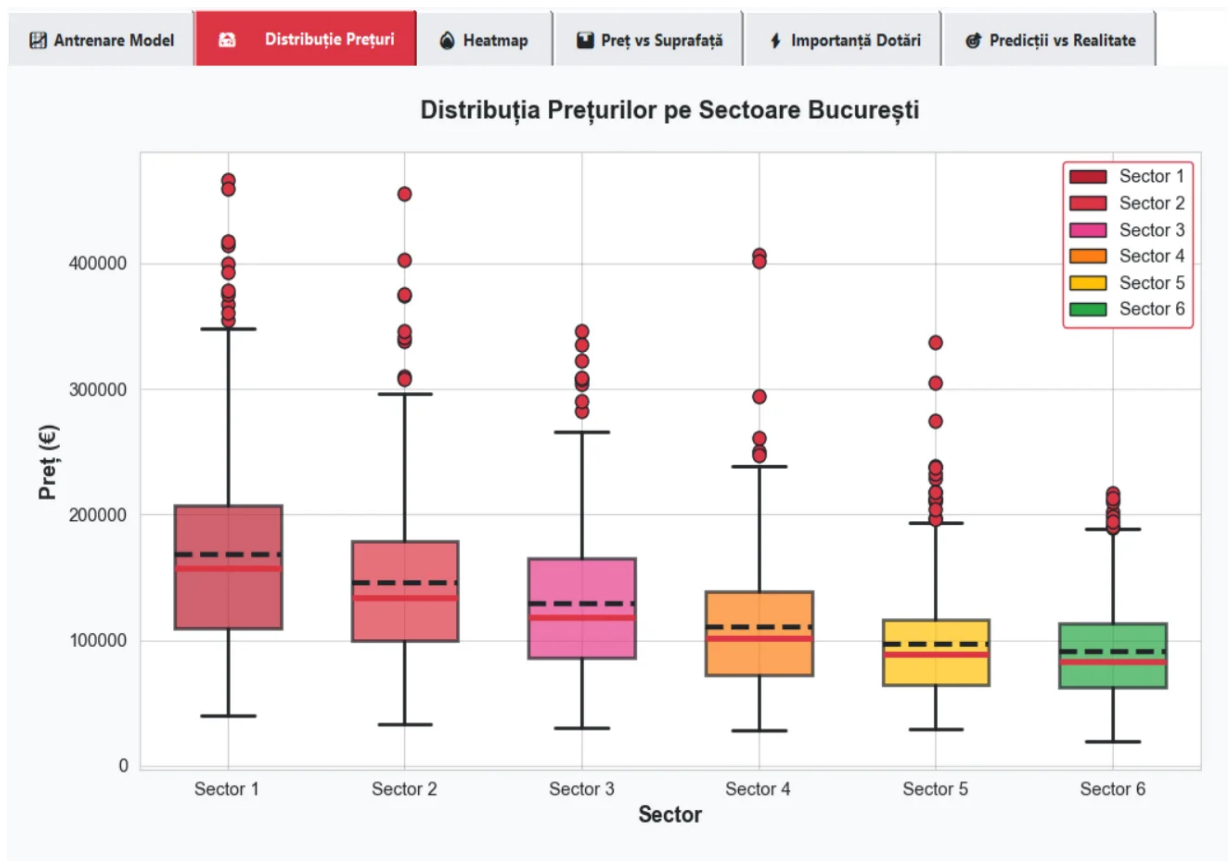


Fig. 6 Distribuția prețurilor pe sectoare București

Din grafic se observă clar ierarhia prețurilor pe sectoare. Sectorul 1 are cele mai mari prețuri, urmat de Sectoarele 2 și 3, apoi Sectoarele 4, 5 și 6 cu prețuri progresiv mai mici. Valorile aberante, reprezentate prin puncte roșii, indică proprietăți excepționale care se abat semnificativ de la distribuția generală a sectorului respectiv.

Graficul Heatmap - Matricea de Corelație

Al treilea grafic este o heatmap care prezintă corelațiile dintre toate variabilele din setul de date. Valorile variază de la -1, reprezentând corelație negativă perfectă în nuanțe de roșu, până la +1, reprezentând corelație pozitivă perfectă în nuanțe de verde.



Fig. 7 Matricea de corelație - Dotări vs Preț

Heatmap-ul oferă o vedere de ansamblu asupra relațiilor dintre variabile. Se observă că suprafața are cea mai puternică corelație pozitivă cu prețul, ceea ce este intuitiv: apartamentele mai mari costă mai mult. Sectorul are o corelație negativă cu prețul, reflectând faptul că sectoarele cu numere mai mari tind să aibă prețuri mai mici. Numărul de camere și anul construcției au corelații pozitive moderate cu prețul.

Graficul Preț vs Suprafață

Al patrulea grafic este un scatter plot care prezintă relația dintre suprafață și preț, cu punctele colorate în funcție de sector. Linia întreruptă roșie reprezintă trendul general, calculat prin regresie liniară.

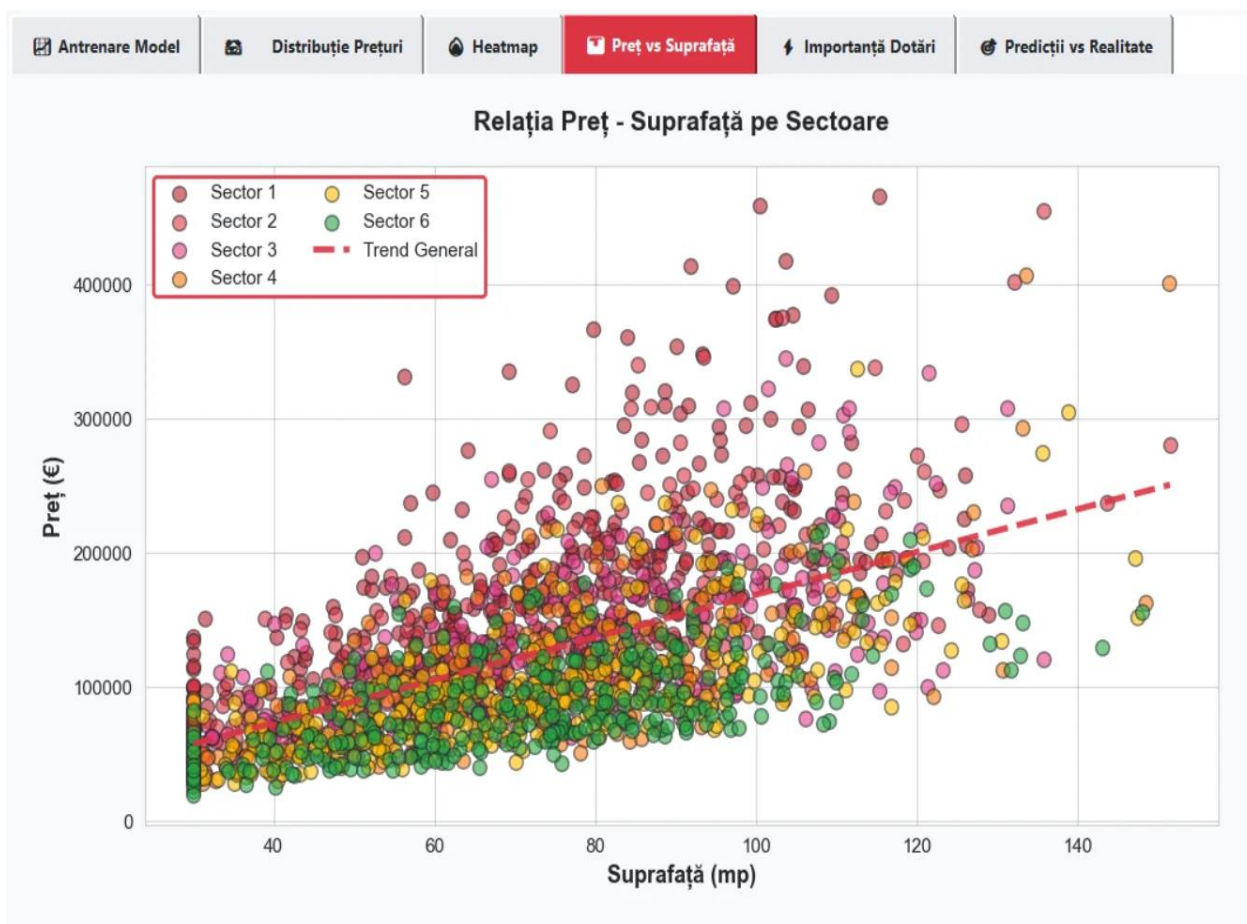


Fig. 8 Relația Preț - Suprafață pe Sectoare

Graficul confirmă corelația pozitivă puternică dintre suprafață și preț. Se observă și stratificarea pe sectoare: pentru aceeași suprafață, proprietățile din Sectoarele 1 și 2 au prețuri mai mari decât cele din Sectoarele 5 și 6. Această vizualizare ajută la înțelegerea interacțiunii dintre suprafață și locație în determinarea prețului.

Graficul Importanță Dotări

Al cincilea grafic este un bar chart orizontal care prezintă importanța fiecărei caracteristici în determinarea prețului, bazată pe coeficienții de corelație absolută cu prețul.

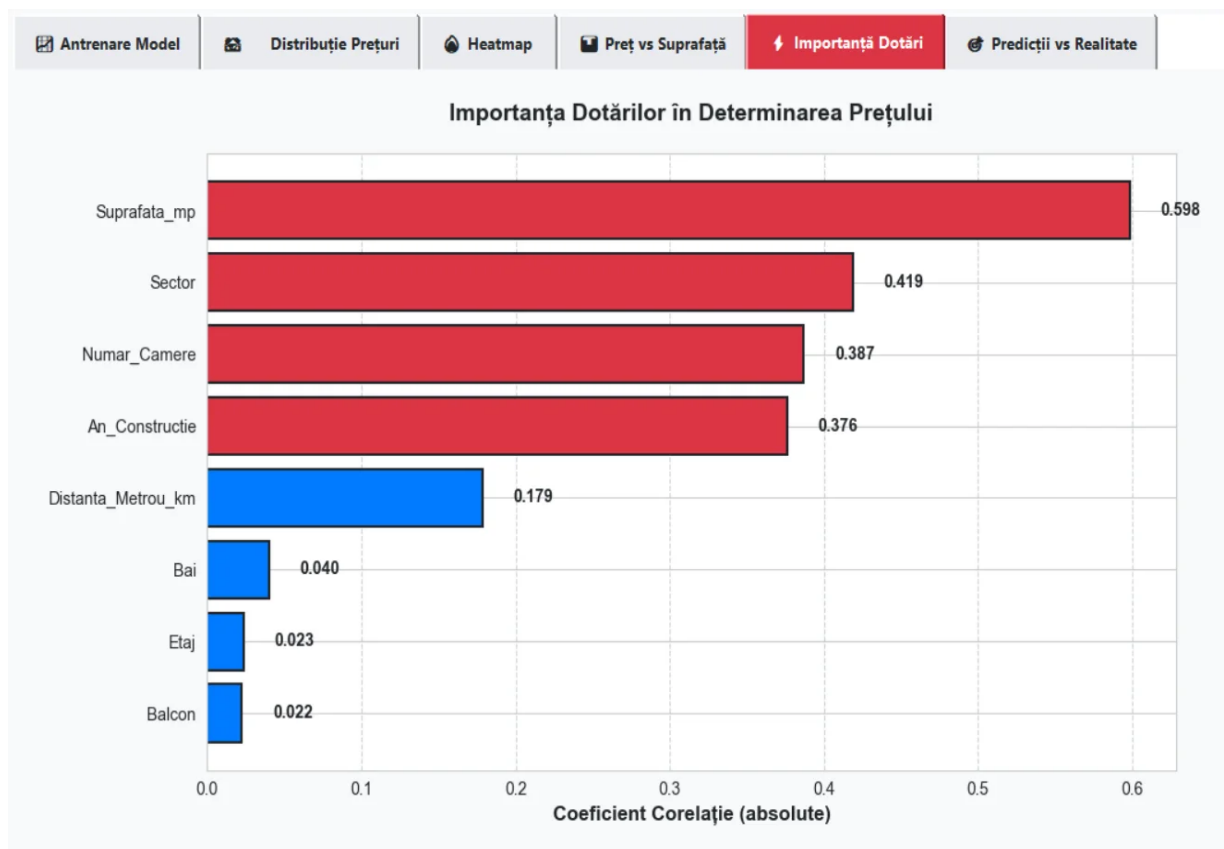


Fig. 9 Importanța dotărilor în determinarea prețului

Din grafic se observă că cele mai importante caracteristici sunt suprafața, sectorul, numărul de camere și anul construcției. Caracteristicile cu importanță mai mică includ distanța până la metrou, numărul de băi, etajul și balconul. Barele roșii indică corelații peste median, iar cele albastre sub median, oferind o perspectivă vizuală imediată asupra importanței relative a fiecărui factor.

Graficul Predicții vs Realitate

Al șaselea grafic este un scatter plot care compară predicțiile modelului cu valorile reale din setul de test. Linia întreruptă neagră reprezintă predicția perfectă, unde predicția ar fi exact egală cu realitatea.

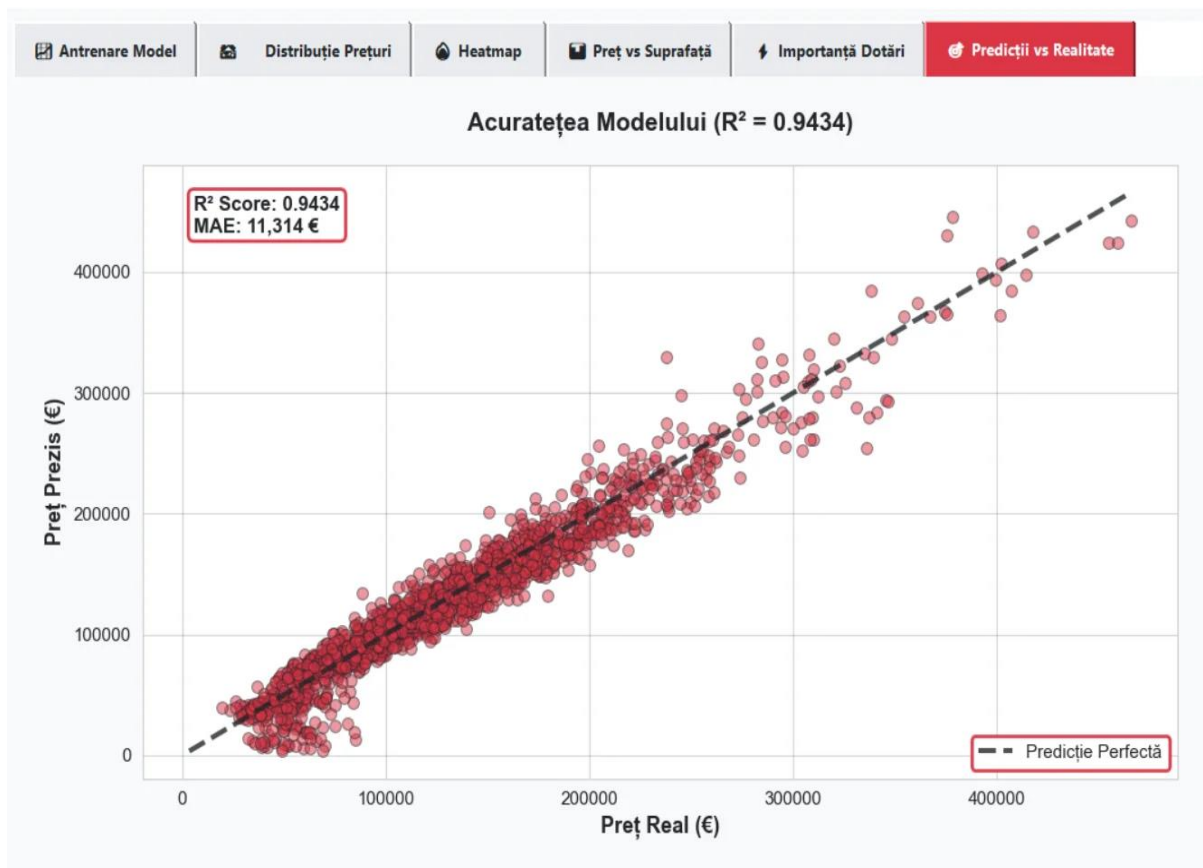


Fig. 10 Acuratețea modelului - Predicții vs Realitate

Cu cât punctele sunt mai apropiate de linia diagonală, cu atât modelul este mai precis. Graficul afișează și valorile R^2 Score și MAE în colțul stânga sus, oferind o evaluare cantitativă a performanței. Dispersia mai mare pentru prețurile mari este normală, deoarece proprietățile scumpe tind să aibă caracteristici mai diverse și sunt mai greu de prezis cu precizie.

4.3. Mesajele din terminal

În timpul rulării aplicației, în terminal sunt afișate mesaje informative despre stadiul procesului de antrenare și performanța modelului. Aceste mesaje sunt esențiale pentru debugging și monitorizarea comportamentului aplicației.

Mesaje la încărcarea modelului existent

Când aplicația pornește și găsește un model salvat anterior, sunt afișate mesaje de confirmare pentru fiecare componentă încărcată: datele din fișierul CSV, modelul salvat, și scaler-ul pentru normalizare.

```
2026-01-12 21:32:12.581666: I tensorflow/core/util/port.cc:113] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING:tensorflow:From C:\Users\Administrator\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

✅ Date încărcate din synthetic data.csv
2026-01-12 21:32:17.701236: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use a available CPU instructions in performance-critical operations.
To enable the following instructions: SSE SSE2 SSE3 SSE4.1 SSE4.2 AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:From C:\Users\Administrator\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\keras\src\backend.py:1398: The name tf.executing_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

✅ Model încărcat din saved_model.keras
✅ Scaler încărcat din scaler.pkl
✅ Model și date încărcate cu succes!
✅ Grafic Predicții vs Realitate generat (R²: 0.9343, MAE: 12,015€)
⚠️ Generare date și antrenare model nou...
Epoch 1/150
```

Fig. 11 Mesaje terminal la încărcarea modelului salvat

Se pot observa și warning-uri de la TensorFlow despre funcții deprecate, care pot fi ignorate în siguranță deoarece nu afectează funcționalitatea. După încărcarea cu succes, aplicația afișează metricile de performanță ale modelului încărcat și generează graficele.

Mesaje la antrenarea unui model nou

Când se reantrenează modelul, fie la prima rulare, fie la apăsarea butonului Reantrenare Model, sunt afișate mesaje detaliate despre fiecare epocă de antrenare.

```
Epoch 80/150
44/44 [=====] - 0s 2ms/step - loss: 852115648.0000 - mae: 22050.6250 - val_loss: 257844288.0000 - val_mae: 11472.5205 - lr: 6.9021e-04
Epoch 81/150
44/44 [=====] - 0s 2ms/step - loss: 943942464.0000 - mae: 23626.4570 - val_loss: 244329344.0000 - val_mae: 11344.5430 - lr: 6.8701e-04
Epoch 82/150
44/44 [=====] - 0s 3ms/step - loss: 876030080.0000 - mae: 22833.8457 - val_loss: 248497792.0000 - val_mae: 11616.1602 - lr: 6.8384e-04
Epoch 83/150
34/44 [=====>.....] - ETA: 0s - loss: 863074432.0000 - mae: 22384.6484Restoring model weights from the end of the best epoch: 68.
44/44 [=====] - 0s 2ms/step - loss: 831598016.0000 - mae: 21929.8398 - val_loss: 254692560.0000 - val_mae: 11610.1104 - lr: 6.8067e-04
Epoch 83: early stopping

📊 Performanță Model:
MSE: 277,272,404.97
MAE: 11,914.14
R² Score: 0.9278
✅ Date salvate în synthetic_data.csv
✅ Model salvat în saved_model.keras
✅ Scaler salvat în scaler.pkl
✅ Feature names salvate
✅ Grafic Predicții vs Realitate generat (R²: 0.9434, MAE: 11,314€)
```

Fig. 12 Mesaje terminal în timpul antrenării

Pentru fiecare epocă sunt afișate numărul epocii, un progress bar, timpul per pas, loss-ul de antrenare, MAE de antrenare, loss-ul de validare și MAE de validare. Când early stopping se activează, un mesaj anunță oprirea antrenării și restaurarea ponderilor de la cea mai bună epocă. La final sunt afișate performanța modelului și confirmarea salvării componentelor.

4.4. Fișierul CSV și structura datelor

Datele sintetice generate de aplicație sunt salvate în fișierul `synthetic_data.csv` pentru persistență între sesiuni. Acest fișier conține 2000 de înregistrări, fiecare reprezentând o proprietate imobiliară cu toate caracteristicile și prețul.

	A	B	C	D	E	F	G	H	I
1	Sector	Suprafata_	Numar_Ca	Etaj	An_Constru	Distanța_M	Bai	Balcon	Pret_Euro
2	4	61.2	3	8	2010	0.9	2	1	103000
3	5	30	3	10	1970	0.16	1	0	32500
4	3	84.3	4	2	2010	1.84	1	1	176800
5	5	79.4	3	4	1979	0.1	1	1	115400
6	5	55.9	1	7	2024	3.06	2	1	83800
7	2	68	3	5	1973	0.22	2	0	147400
8	3	37.2	3	3	2004	0.2	2	0	90800
9	3	100	4	7	1979	0.29	1	1	176200
10	3	57.8	4	10	2016	4.84	2	1	120800
11	5	147.4	3	2	1980	1.77	1	1	152300
12	4	65.6	2	2	1983	2.42	1	1	63800
13	3	51	4	3	2007	0.76	1	1	118500
14	6	59.7	5	8	1981	0.61	1	1	64800
15	5	34.7	5	4	2014	0.4	2	1	112200
16	2	82.1	1	7	1986	0.98	1	0	107200
17	4	59.5	3	5	1985	1.2	1	1	71300
18	6	98.3	2	2	2004	1.5	1	1	111400
19	6	63.1	1	7	1971	0.45	1	1	54400
20	2	104.6	3	8	1980	4.45	1	1	117200
21	4	88.9	4	7	2010	0.28	1	1	221700
22	5	70.7	5	3	1972	5	1	1	84200
23	1	109.5	2	10	1998	0.1	2	1	229000
24	4	93.8	5	5	1987	0.98	2	1	169800
25	2	143.7	4	0	1989	0.61	2	1	237600
26	6	59.8	3	3	2016	0.89	1	1	96500

Fig. 13 Structura fișierului CSV cu datele sintetice

Coloanele din fișierul CSV includ Sector cu valori de la 1 la 6, Suprafața în metri pătrați, Numărul de Camere, Etaj, Anul Construcției, Distanța până la Metrou în kilometri, Numărul de Băi, Balcon ca valoare binară și Prețul în Euro. Datele pot fi deschise în Excel sau importate în alte aplicații pentru analiză suplimentară.

5. Explicarea importurilor și bibliotecilor

În această secțiune vom detalia toate importurile utilizate în codul aplicației și rolul fiecărei biblioteci în funcționarea sistemului.

Importul **numpy** as **np** aduce biblioteca fundamentală pentru calcul numeric în Python. În proiect, NumPy este folosit pentru generarea distribuțiilor statistice precum distribuția normală și exponențială, pentru operații condiționale pe array-uri, pentru calculul regresiei liniare pentru linia de trend și pentru crearea array-urilor pentru grafice.

Importul **pandas** as **pd** aduce biblioteca pentru manipularea datelor tabelare. Pandas creează DataFrame-ul cu datele sintetice, calculează statisticile pe sectoare folosind funcții precum groupby, mean și median, generează matricea de corelație și gestionează salvarea și încărcarea datelor în format CSV.

Importul **matplotlib.pyplot** as **plt** aduce biblioteca principală de vizualizare. Matplotlib este folosit pentru crearea tuturor celor șase grafice din aplicație: line plot pentru evoluția antrenării, boxplot pentru distribuția prețurilor, heatmap pentru corelații, scatter plot pentru relația preț-suprafață, bar chart pentru importanța features și scatter plot pentru predicții versus realitate.

Importul **FigureCanvasTkAgg** din `matplotlib.backends.backend_tkagg` permite integrarea figurilor Matplotlib în interfața Tkinter. Această componentă creează un widget canvas care poate afișa grafice Matplotlib într-o fereastră Tkinter, permițând vizualizarea interactivă a graficelor în aplicație.

Importul **Figure** din **matplotlib.figure** aduce clasa container de nivel înalt pentru toate elementele unui grafic. În proiect, se creează câte o Figure pentru fiecare tab de vizualizare, cu dimensiuni și culori de fundal specificate.

Importul **seaborn** as **sns** aduce biblioteca pentru vizualizare statistică avansată. Seaborn este folosit pentru setarea stilului vizual al graficelor cu tema whitegrid care adaugă o grilă de fundal pentru îmbunătățirea lizibilității.

Importurile **tensorflow** și **keras** formează nucleul de machine learning al aplicației. Acestea sunt folosite pentru definirea arhitecturii rețelei neuronale cu straturi Sequential, Dense, BatchNormalization și Dropout, pentru compilarea modelului cu optimizator Adam și loss MSE, pentru antrenarea cu callbacks EarlyStopping și ReduceLROnPlateau, pentru salvarea și încărcarea modelului și pentru efectuarea predicțiilor.

Importul **train_test_split** din **sklearn.model_selection** împarte datele în seturi de antrenare, validare și test. Parametrul `test_size` controlează proporția, iar `random_state` asigură reproducibilitatea experimentelor.

Importul **StandardScaler** din **sklearn.preprocessing** normalizează features-urile prin eliminarea mediei și scalarea la varianță unitară. Metodele `fit_transform` și `transform` sunt

folosite pentru a normaliza datele de antrenare și apoi pentru a aplica aceeași transformare pe datele de validare și test.

Importurile **r2_score**, **mean_absolute_error** și **mean_squared_error** din **sklearn.metrics** calculează metricile de evaluare a modelului. Acestea măsoară coeficientul de determinare, eroarea medie absolută în Euro și eroarea medie pătratică folosită ca funcție de loss.

Importurile **tkinter** și **modulele sale** sunt folosite pentru întreaga interfață grafică. Modulul principal tk oferă widget-uri de bază precum Frame, Label, Button, Canvas, Scale și Text. Modulul ttk oferă widget-uri stilizate precum Notebook pentru tab-uri. Modulul messagebox afișează dialoguri de confirmare și eroare, iar filedialog gestionează dialogul de export CSV.

Importurile **json**, **os** și **pickle** sunt module standard Python. Modulul json salvează și încarcă numele features într-un fișier JSON. Modulul os verifică existența fișierelor și gestionează căile. Modulul pickle serializează și deserializează obiectul StandardScaler pentru persistență între sesiuni.

Importul **datetime** din **modulul datetime** generează timestamp-uri în istoricul predicțiilor și pentru denumirea fișierelor de export cu data și ora curentă.

Importul **stats** din **scipy** oferă funcții statistice avansate. Deși este importat, în versiunea curentă a proiectului este folosit minimal, dar este disponibil pentru extinderi viitoare ale analizei statistice.

Importul **warnings** și apelul **warnings.filterwarnings** cu parametrul ignore suprimă afișarea warning-urilor în consolă, în special a celor generate de TensorFlow referitoare la funcții deprecate. Acest lucru menține output-ul consolei curat și concentrat pe mesajele relevante.

6. Concluzie

Concluzia acestui proiect evidențiază complexitatea, utilitatea și aplicabilitatea reală a unei aplicații de predicție a prețurilor imobiliare bazate pe inteligență artificială. Proiectul UrbanPrice Bucharest reunește toate componentele esențiale ale unei aplicații moderne de machine learning, oferind funcționalități robuste și o experiență utilizator intuitivă.

Pe parcursul dezvoltării, au fost aplicate concepte fundamentale din domeniul inteligenței artificiale. Am implementat generarea de date sintetice realiste bazate pe cunoașterea domeniului imobiliar din București, proiectarea unei arhitecturi de rețea neuronală cu regularizare pentru prevenirea overfitting-ului, implementarea unui pipeline complet de antrenare cu callbacks pentru optimizarea performanței și evaluarea modelului cu metrice standard.

Modelul de rețea neuronală dezvoltat atinge o acuratețe ridicată, explicând o proporție semnificativă din variația prețurilor imobiliare. Eroarea medie absolută obținută este acceptabilă în contextul unei piețe cu prețuri care variază pe o gamă largă, demonstrând capacitatea modelului de a face predicții utile în practică.

Interfața grafică dezvoltată în Tkinter oferă o experiență completă utilizatorului. Aceasta permite introducerea intuitivă a caracteristicilor proprietății prin slider-uri, vizualizarea rezultatelor cu comparații de piață, acces la statistici detaliate pe sectoare, istoric al predicțiilor cu posibilitate de export și șase grafice interactive pentru analiza datelor și performanței modelului.

Proiectul demonstrează aplicabilitatea practică a tehnicilor de machine learning în rezolvarea problemelor din lumea reală. Deși utilizează date sintetice, metodologia poate fi aplicată direct pe date reale când acestea devin disponibile. Arhitectura modulară a codului permite extinderea ușoară cu funcționalități noi precum integrarea cu API-uri de date imobiliare reale, adăugarea de features suplimentare, implementarea altor algoritmi de machine learning pentru comparație sau deployment-ul ca aplicație web.

În concluzie, acest proiect nu doar că îndeplinește obiectivele inițiale, ci demonstrează o înțelegere solidă a întregului ciclu de dezvoltare a aplicațiilor de inteligență artificială și oferă o soluție funcțională, modernă și bine structurată pentru estimarea prețurilor imobiliare din București.

7. Bibliografie

1. <https://www.tensorflow.org/guide> - Documentație oficială TensorFlow și Keras
2. <https://scikit-learn.org/stable/documentation.html> - Documentație Scikit-learn
3. <https://pandas.pydata.org/docs/> - Documentație Pandas
4. <https://matplotlib.org/stable/contents.html> - Documentație Matplotlib
5. <https://numpy.org/doc/> - Documentație NumPy
6. <https://docs.python.org/3/library/tkinter.html> - Documentație Tkinter
7. Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition. O'Reilly Media.
8. Chollet, F. (2021). Deep Learning with Python, 2nd Edition. Manning Publications.